PNI ソフトウェア健全性診断レポート

対象プロジェクト `order-dispatch` (`v1.2.0`) に対する PNI (Project Normalized Inspection) ステートフル診断レポートです。静的 AST 構造解析・アーキテクチャ境界検証・コンテナ隔離下の動的テスト／カバレッジ計測・Trivy 脆弱性スキャン・動的 Web ペネトレーションテスト（`web_pentest`）を統合し、7軸のエンジニアリング健全性を定量評価しています。

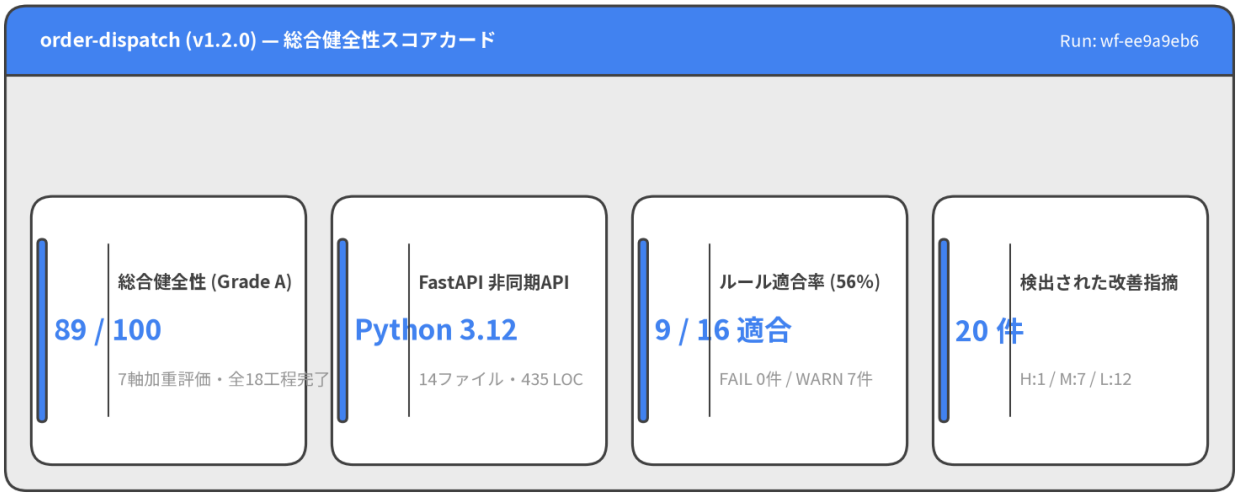


図0-1: order-dispatch 総合健全性スコアカード (89/100・Grade A)

診断メタデータ概要

項目	診断記録内容
対象プロジェクト / バージョン	`order-dispatch` (`1.2.0`) — 顧客注文受付・整数セント価格計算・非同期パートナー Webhook 配送サービス
対象リポジトリ相対パス	`examples/python/api_fastapi_clean` (`src/order_dispatch/**` 14ファイル、 `tests/**` 3ファイル)
主要技術スタック	Python `>=3.11` (`python:3.12.6-slim-bookworm`)、FastAPI `>=0.115.0` 、Pydantic v2、httpx、uvicorn、uv
実行ワークフロー / Run ID	`py_api_fastapi_clean` / `wf-ee9a9eb6` （全18ステップ完遂・状態DB記録済）
診断実行日時 (UTC)	`2026-10-24T14:48:14Z`
コンテナ動的検証結果	`passed` — Dockerfileビルド成功 (`28.40s`)、オフライン単体テスト `4/4` 成功 (`5.40s`)、行カバレッジ `85.2%` 、 `/healthz` HTTP `200` (`2.49s`)
セキュリティ・DAST検査結果	`passed` — Python実行時依存CVE `0件` （ `pip@24.2` 6件、Debian 12.7 OS 508件)、動的侵入検証 `15/19` 合格（CRIT/HIGH `0件` 、MED/LOW `3件` ）
総合評価 (Overall Grade)	**89 / 100 (Grade A)** — クリーン4層設計・型安全性100%・CWE危険シンク0件・ライブ注入耐性を達成。基盤イメージ刷新とAPI認証追加が優先課題

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・API境界

1. エグゼクティブサマリー

`order-dispatch`（`v1.2.0`）の総合健全性スコアは **89 / 100 (Grade A)** です。FastAPI Lifespan DI と Repository Protocol による厳格な4層クリーンアーキテクチャ、型アノテーション・Docstring・Import衛生すべて `100.0%`、CWE 危険コードシンク `0件`、Python アプリケーション実行時ライブラリの CVE `0件` を達成しており、さらに隔離コンテナに対する動的 Web ペネトレーションテスト（`web_pentest`：全19プローブ中15件合格・CRITICAL/HIGH `0件`）でも Pydantic `extra="forbid"` によるマスアサインメント拒否（HTTP `422`）および XSS・SQLi・パストラバーサル耐性が実測確認されました。最優先改善（P0〜P1）は、`python:3.12.6-slim-bookworm` ベースイメージ刷新、ライブプローブでも実証された `POST /v1/orders`（HTTP `201`、`CWE-306`）への OAuth2/JWT 認証・認可導入、`uv.lock` の完全ロック化、および ASGI HTTP 統合テスト拡充による分岐カバレッジ（現状 `62.5%`）の引き上げです。

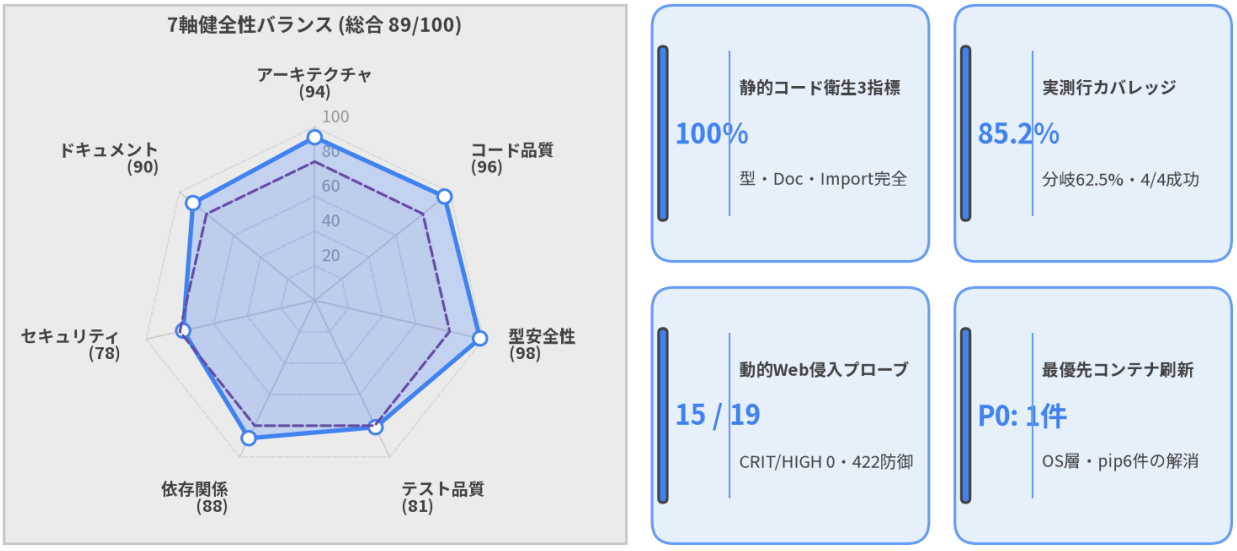


図1-1: 7軸エンジニアリング健全性レーダーチャートと主要品質KPI

主要な強みと優先リスクの対比

静的構造・サプライチェーン統制・12-Factor 運用準備度はいずれも高水準（監査ステップスコア `91〜93`）である一方、コンテナ OS ベースイメージの経年脆弱性・ライブ DAST で実証された `/v1/orders` 未認証注文生成（`CWE-306`）や HTTP ヘッダ強化・HTTP ルート層の実測分岐カバレッジに改善余地が集中しています。

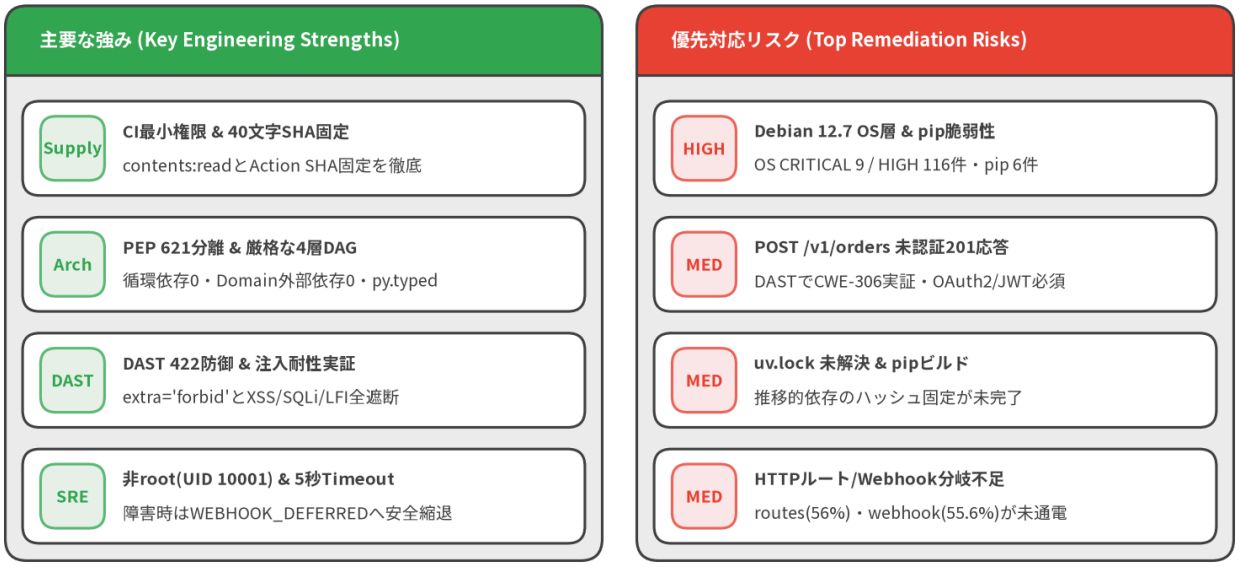


図1-2: 確認されたエンジニアリング上の強み（左）と優先対応リスク（右）

7軸評価サマリー

評価軸	スコア	評価根拠の要点
型安全性 (`typing`)	**98**	全23関数・14クラスで型注釈率 `100.0%`、`Any` や無制限 `type: ignore` `0件`、PEP 561 `py.typed` 同梱。
コード品質 (`code_quality`)	**96**	Google形式 Docstring `100.0%`、Import衛生 `100.0%`、複雑関数 `0件`、グローバル可変状態 `0件`。
アーキテクチャ (`architecture`)	**94**	API → Services → Domain / Infra の単方向 DAG（循環依存 `0件`）。Lifespan HTTP 接続プール共有でさらに向上。
ドキュメント (`documentation`)	**90**	`README.md` と `SECURITY.md` で起動・設定・API例を明記。開発者向け検証コマンド（`pytest` / `ruff` / `ty`）追記を推奨。
依存関係 (`dependencies`)	**88**	実行時依存5件と `dev` グループを明確分離。`uv.lock` の完全解決とマルチステージ Docker 化が残課題。
テスト品質 (`testing`)	**81**	単体テスト `4/4` がオフライン環境で確定成功（行 `85.2%`）。`TestClient` / `MockTransport` 追加で分岐 `62.5%` を改善可能。
セキュリティ (`security`)	**78**	実行時依存CVE `0件`・危険シンク `0件`・DAST重大脆弱性 `0件`（`15/19` 合格）。OSパッチ適用と `/v1/orders` 認証（`CWE-306`）・ヘッダ強化が課題。

2. アーキテクチャとソースコード構成

`order-dispatch` は **FastAPI Lifespan 依存性注入（DI）と Repository Protocol ポートを備えた4層クリーンアーキテクチャ** を採用する非同期 Web API マイクロサービス（`runtime=web_api`、全14ファイル・435 LOC）です。内部 Import グラフは完全な有向非巡回グラフ（循環依存 `0件` ・レイヤ境界違反 `0件` ）であり、`domain/` パッケージは FastAPI・Pydantic・httpx への依存を一切持ちません。



アーキテクチャ層	LOC	主要コンポーネントと許可された依存方向
Composition Root	68	main.py, settings.py — Lifespan DI配線・設定検証
API Transport	143	routes.py, schemas.py -> Services / Domain
Application Svc	76	OrderDispatcherService -> Domain / Repository
Infra Adapters	76	OrderRepository, AsyncWebhookClient -> Domain
Domain Core	72	OrderRecord, OrderLine, pricing -> 外部依存ゼロ

図2-1: 物理ソースディレクトリ構造 (TreeNode) とレイヤ別責務・規模サマリー (Table)

レイヤ依存関係とリスクホットスポット・実行フロー

以下のクラス／コンポーネント依存図では、`module_hotspots` の実測リスクスコアをサーマルオーバーレイ（`HeatmapOverlay` ）として重畳しています。アプリケーション内部コードの複雑度リスクは極めて低い一方、リスクはコンテナ基盤 `Dockerfile` （ `0.70` ）、認証境界の未実装が動的検証でも確認された `api/routes.py` （ `0.60` ）、および HTTP ヘッダ／OpenAPI 公開制御を担う `main.py` （ `0.18` ）に局在しています。

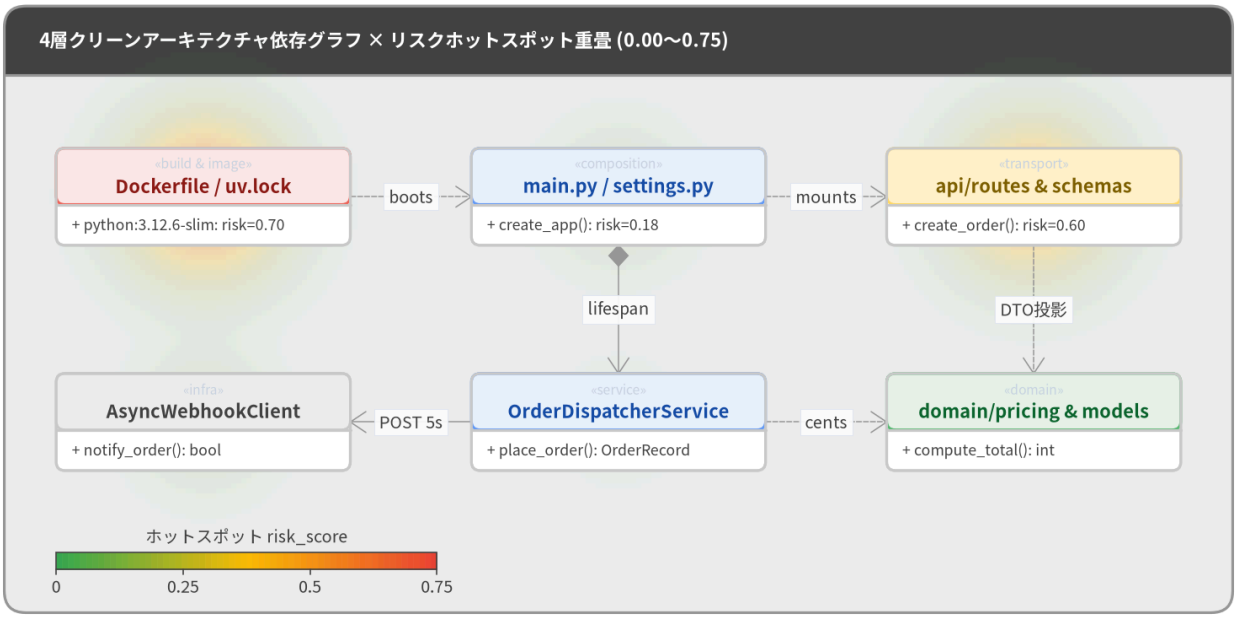


図2-2: レイヤ間コンポーネント依存関係とモジュール別リスクホットスポット (HeatmapOverlay)

注文作成リクエスト（`POST /v1/orders`）の実行時ライフサイクルは、DTO 検証・整数セント価格計算・5秒タイムアウト付き非同期 Webhook 通知・状態遷移（`DISPATCHED` または `WEBHOOK_DEFERRED`）の5段階で完結します。



図2-3: 注文作成・価格計算・非同期Webhook配送の5段階実行フロー

全ソースモジュール一覧（14ファイル・435 LOC）

モジュール相対パス	LOC	公開クラス / 型	公開関数・エンドポイント / 役割
`src/order_dispatch/main.py`	48	—	`lifespan`, `create_app` — ASGI アプリ生成と Lifespan DI 初期化
`src/order_dispatch/settings.py`	18	`ServiceSettings`	`load_settings` — `ORDER_DISPATCH_*` 環境変数の型検証
`src/order_dispatch/api/routes.py`	74	—	`health_check`, `create_order`, `get_order`, `get_dispatcher`, `get_settings`
`src/order_dispatch/api/schemas.py`	67	`OrderLineInput`, `CreateOrderRequest`, `OrderResponse`, `HealthResponse`	`to_domain_lines`, `to_order_response` (`extra="forbid"` DTO変換)
`src/order_dispatch/services/dispatcher.py`	74	`OrderDispatcherService`	`place_order`, `find_order` — 価格算出・Webhook通知・保存の調停
`src/order_dispatch/domain/models.py`	29	`CustomerTier`, `OrderStatus`, `OrderLine`, `OrderRecord`	不変 `frozen=True` エンティティとライフサイクル `StrEnum`
`src/order_dispatch/domain/pricing.py`	41	`PricingError`	`calculate_subtotal_cents`, `compute_order_total_cents`（整数セント演算）
`src/order_dispatch/infra/repository.py`	39	`OrderRepository`, `InMemoryOrderRepository`	`Protocol` 永続化ポートとインメモリ辞書アダプタ
`src/order_dispatch/infra/webhook_client.py`	35	`AsyncWebhookClient`	`notify_order_accepted`（5秒タイムアウト・例外時 `False` 縮退）
`src/order_dispatch/**/__init__.py` (計5件)	10	—	パッケージ境界初期化モジュール（各2 LOC × 5ファイル）

3. 推奨する変更対応

検出された20件の指摘と7件の `WARN` ルールを最小工数で解消するため、緊急度・効果順に **P0（即時対応：1件）→ P1（高優先度：4件）→ P2（本番運用・HTTPヘッダ強化：6件）** の計11アクションへ集約しました。対象コードの構造変更は小さく、`Dockerfile`・`uv.lock`・API認証依存関数・ASGI 統合テスト・セキュリティヘッダミドルウェアの整備で全リスクを解消できます。

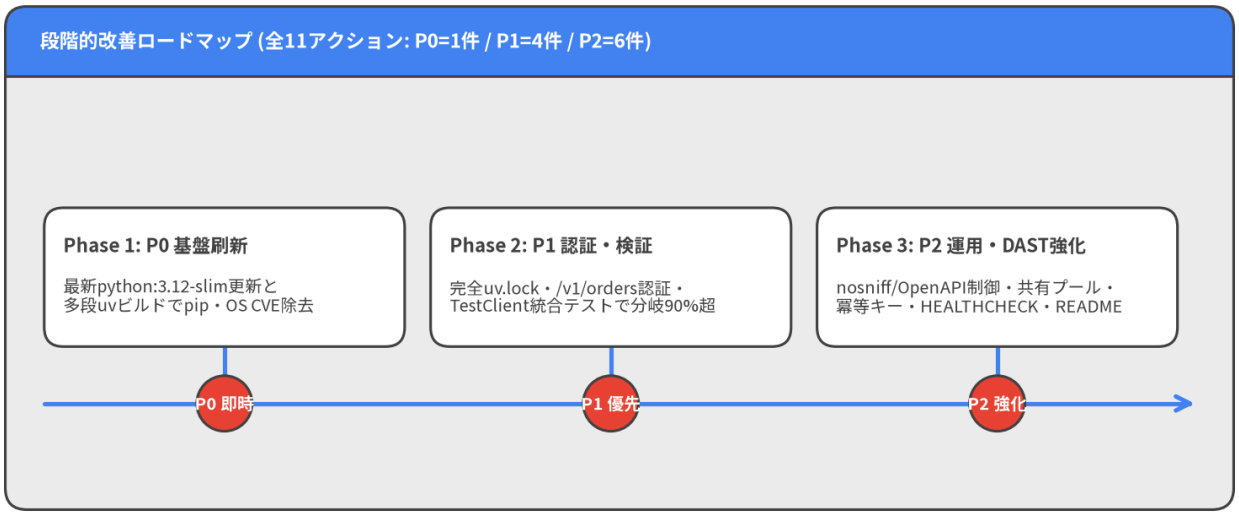


図3-1: フェーズ別改善ロードマップ (P0 基盤脆弱性解消 -> P1 認証・ロック・検証強化 -> P2 本番運用・DASTヘッダ強化)

優先アクションと期待効果のマトリクス

上位5件（P0〜P1）および DAST 指摘（`CWE-306` / `CWE-200` / `CWE-693`）の完了により、セキュリティ軸（`78` → `94+`）・テスト軸（`81` → `95+`）・依存関係軸（`88` → `96+`）が大幅に改善し、総合 Grade S 水準（93点以上）へ到達します。

優先度	推奨アクション要点	変更対象ファイル	解消されるリスク / 期待効果
P0	ベースイメージ更新 & pipレス多段ビルド	Dockerfile	pip CVE 6件全廃・OpenSSL/GnuTLS CRITICAL解消
P1	完全な uv.lock 生成 & --frozen 導入	uv.lock, Dockerfile, ci.yml	SHA-256ハッシュ検証済の100%決定論的ビルド
P1	/v1/orders OAuth2/JWT 認証・認可強制	api/routes.py, settings.py	DAST CWE-306未認証201注文注入・IDOR参照を防止
P1	TestClient + Lifespan HTTP 統合テスト	tests/test_routes.py	routes.py(56%)・main.py(67.9%)の未実行経路解消
P1	AsyncWebhookClient MockTransport 検証	tests/test_dispatcher.py	分岐カバレッジを62.5%から90%超へ引き上げ

図3-2: P0〜P1 最優先改善アクションの対象ファイル・関連ルール・期待効果サマリー

全11件の改善アクション一覧（トレーサビリティ表）

#	優先度	分野	推奨アクションと具体実装内容	対象ファイル
1	P0	`security`	`python:3.12-slim-bookworm` **最新化とマルチステージ** `uv` **ビルド採用**: 最新パッチ済イメージへ更新し、本番ステージから `pip@24.2` を除外する。	`Dockerfile`
2	P1	`dependencies`	**完全な** `uv.lock` **再生成と** `--frozen` **インストールの徹底**: `uv lock` で推移的依存とハッシュを記録し、`pip install .` を `uv sync --frozen --no-dev` に置換。	`uv.lock`, `Dockerfile`, `.github/workflows/ci.yml`
3	P1	`security`	`/v1/orders` **エンドポイントへの OAuth2/JWT 認証・テナント認可追加**: FastAPI `Depends` で Bearer トークンと `customer_id` 権限を検証し、未認証 HTTP `201`（`CWE-306`）を遮断。	`src/order_dispatch/api/routes.py`, `src/order_dispatch/settings.py`
4	P1	`testing`	**FastAPI ルート・Lifespan の ASGI HTTP 統合テスト追加**: `with TestClient(create_app()) as client:` で `200` / `201` / `404` / `422` を検証する。	`tests/test_routes.py`, `tests/test_dispatcher.py`
5	P1	`testing`	**実測分岐・行カバレッジ未達箇所の解消**: `httpx.MockTransport` による `AsyncWebhookClient` 正常・5xx・Timeout 検証と `pricing.py:L49` 境界検証を追加。	`tests/test_routes.py`, `tests/test_dispatcher.py`, `src/order_dispatch/api/routes.py`, `src/order_dispatch/infra/webhook_client.py`
6	P2	`architecture`	**Webhook** `Idempotency-Key` **付与と永続 Repository 切替対応**: `record.order_id` を冪等キーヘッダに設定し、外部 DB アダプタ注入を可能にする。	`src/order_dispatch/infra/webhook_client.py`, `src/order_dispatch/main.py`
7	P2	`architecture`	**Lifespan スコープの** `httpx.AsyncClient` **コネクションプール共有**: リクエスト毎のクライアント生成を廃止し、TLS ハンドシェイク負荷を削減する。	`src/order_dispatch/main.py`, `src/order_dispatch/infra/webhook_client.py`
8	P2	`dependencies`	**コンテナ** `HEALTHCHECK` **命令追加とベースイメージ** `@sha256` **固定**: `/healthz` プローブ定義（Trivy `DS-0026` 解消）とダイジェスト固定を行う。	`Dockerfile`

#	優先度	分野	推奨アクションと具体実装内容	対象ファイル
9	P2	`documentation`	**開発者検証コマンドと全環境変数リファレンスの `README.md` 追記**: `pytest` / `ruff` / `ty` 実行手順および `ORDER_DISPATCH_VIP_DISCOUNT_RATE` を明記する。	`README.md`
10	P2	`security`	**ASGI セキュリティヘッダミドルウェア追加と本番 `/openapi.json` 非公開化**: `X-Content-Type-Options: nosniff` / `X-Frame-Options: DENY` 付与と `settings.env != 'dev'` での OpenAPI 無効化を行い、DAST 全19プローブ通過を達成する。	`src/order_dispatch/main.py`, `src/order_dispatch/settings.py`
11	P2	`security`	**`ServiceSettings` の URL スキーム・数値境界バリデーション強化**: `HttpUrl`（HTTPS必須）と `Field(gt=0.0, le=30.0)` 等の起動時制約を追加する。	`src/order_dispatch/settings.py`

4. 検査指標・ルール適合マトリクスと指摘一覧

本ワークフロー（`py_api_fastapi_clean` / `wf-ee9a9eb6`）では、8つの AI 監査ステップと5つの決定論的検査（`project_inspect`・`ast_analyze`・`run_check`・`security_scan`・`web_pentest`）を通じて全16ルールを評価しました。結果は `FAIL` **0件**・`WARN` **7件**・`PASS` **9件（適合率 56.3%）**、改善指摘は計20件（`HIGH` 1件・`MEDIUM` 7件・`LOW` 12件）です。

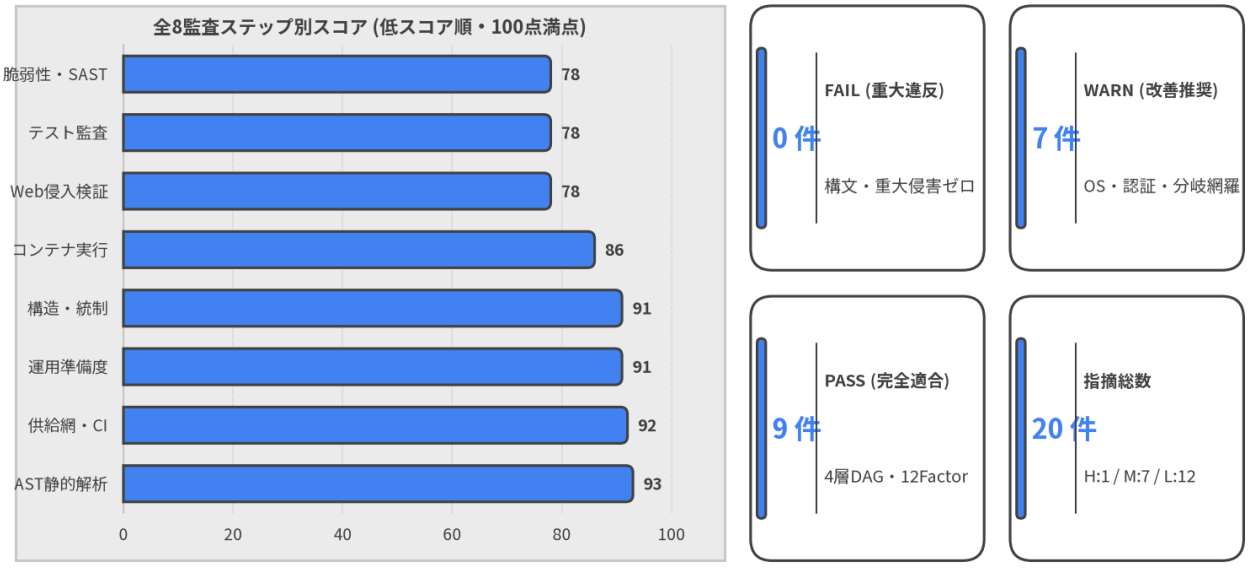


図4-1: 全8監査ステップ別スコア比較（低スコア順）とルール適合・指摘内訳KPI

WARN ルール7件の要点と上位重大度指摘の可視化

`WARN` 判定となった7ルールはいずれも設計破綻ではなく、ベースイメージ更新・ロックファイル完全化・OAuth2/JWT 認証フック＆HTTPヘッダ実装・`TestClient` 統合テスト追加によって確実に `PASS` へ昇格可能です。

判定	ルールID	主な検出証跡 (Evidence)	PASS昇格への解消条件
WARN	common/security_vulnerability	OS 508・pip 6・DAST CWE-306/200/693	最新slim化・API認証・nosniffヘッダ追加
WARN	common/security	routes.py:L60,L83 認証・認可未実装	/v1/ordersへのOAuth2/JWT認可強制
WARN	python/dependencies	uv.lock推移的依存未記録・pip install	uv lock完全化とuv sync --frozen適用
WARN	common/supply_chain	Dockerfile HEALTHCHECK欠落・pip残存	HEALTHCHECK追加とマルチステージ化
WARN	python/testing	test_routes.pyがHTTP呼び出し未実行	TestClient + MockTransportテスト追加
WARN	common/runtime_verification	全体分岐62.5% (routes 56%, webhook 55.6%)	HTTP全分岐通電で分岐75%以上へ向上
WARN	common/documentation	READMEにpytest/ruff/ty手順が未記載	開発検証コマンドと全環境変数を追記

図4-2: 要注意 (WARN) ルール7件の証跡サマリーと解消アプローチ

全16ルール適合マトリクス（`WARN` → `PASS` 順）

判定	ルールID	Step	主な検証証跡と判定根拠
WARN	`common/documentation`	`act-02`	`README.md:L1-L34` に概要・起動・設定・API例を備えるが、`pytest` / `ruff` / `ty` 実行手順と `ORDER_DISPATCH_VIP_DISCOUNT_RATE` 記載が不足。
WARN	`common/runtime_verification`	`act-12`	ビルド(`28.40s`)・単体テスト(`4/4`)・`/healthz` スモーク(`200`)は全て成功、行網羅率 `85.2%` だが分岐網羅率が `62.5%`（目標70%未満）。
WARN	`common/security`	`act-03`	DTO `extra="forbid"` ・非root実行・秘密漏洩0件を満たすが、`routes.py:L60,L73,L83` の OAuth2/JWT 認証・テナント認可が未実装。
WARN	`common/security_vulnerability`	`act-05` / `16`	実行時依存CVE `0件` ・危険シンク `0件` ・DAST重大指摘 `0件` (`15/19` 合格)だが、OS CVE `508件` ・ `pip@24.2` `6件` ・ライブ `CWE-306` / `200` / `693` を検出。
WARN	`common/supply_chain`	`act-14`	GitHub Actions 40文字SHA固定・`contents: read` ・Dependabot週次監視を完備するが、 `Dockerfile` の `pip install .` と `uv.lock` 不完全が残存。
WARN	`python/dependencies`	`act-02`	`pyproject.toml` で実行時5依存と `dev` 群を分離済だが、`uv.lock` にサードパーティ解決ピンとハッシュが含まれていない。
WARN	`python/testing`	`act-10`	テスト4件すべてアサーション有・オフライン決定論的だが、`tests/test_routes.py` が `TestClient` 経由の HTTP 実行を行っていない。
PASS	`common/architecture`	`act-05`	循環依存 `0件` ・境界違反 `0件` 。 `api -> services -> domain/infra` の単方向 DAG と `domain/` のフレームワーク非依存を維持。
PASS	`common/git_workflow`	`act-02`	`.gitignore` でキャッシュ・仮想環境を除外し、 `.github/workflows/ci.yml` で `push` / `pull_request` 自動検証ゲートを運用。
PASS	`common/maintainability`	`act-05`	複雑関数 `0件` ・過剰引数 `0件` ・グローバル状態 `0件` 。最大ファイル長 `74 LOC` （計435 LOC）で高い凝集度と保守性を維持。
PASS	`common/production_readiness`	`act-09`	外部通信タイムアウト漏れ `0件` （ `5.0s` 明示）・障害時 `WEBHOOK_DEFERRED` 縮退・ `/healthz` 起動スモーク `2.49s` で HTTP `200` 応答。
PASS	`common/twelve_factor`	`act-09`	環境変数参照を `ServiceSettings` に一元化（散在参照 `0件` ）、標準 `logging` モジュール採用（ `print` 出力 `0件` ）。
PASS	`python/code_quality`	`act-05`	Google スタイル Docstring カバレッジ `100.0%` 、空 Docstring・シグネチャ不一致ともに `0件` 。

判定	ルールID	Step	主な検証証跡と判定根拠
PASS	`python/frameworks/fastapi`	`act-05`	非推奨 `@app.on_event` 不使用、`@asynccontextmanager lifespan`・`Depends` DI・全ルートの `response_model` 明示を徹底。
PASS	`python/modern_stack`	`act-05`	Import衛生 `100.0%`、ワイルドカード Import `0件`、未使用・未宣言依存 `0件`、`StrEnum` / PEP 604 型記法を採用。
PASS	`python/typing`	`act-05`	型安全性カバレッジ `100.0%`、`Any` 乱用・無制限 `type: ignore` `0件`、`src/order_dispatch/py.typed` マーカー配置済。

全20件の指摘一覧（重大度 `HIGH` → `MEDIUM` → `LOW` 順）

#	重大度	ルールID	箇所	検出された証跡と推奨修正内容
1	HIGH	`common/security_vulnerability`	`Dockerfile:1`	`python:3.12.6-slim-bookworm` (Debian 12.7) に OS CVE 508件（CRITICAL 9 / HIGH 116：`openssl`, `libgnutls30`, `perl-bas` 等）。最新パッチ済イメージへ更新
2	MEDIUM	`python/dependencies`	`uv.lock:4`	ルート `order-dispatch` のみ記載れ推移的依存ピンが欠落、`Dockerfile:9` が `pip install .` 使用。`uv lock` と `uv sync --frozen --no-dev` へ統一。
3	MEDIUM	`common/security`	`api/routes.py:73`	`# TODO(security)` の通り OAuth2/JWT 認証・テナント別レト制限が未実装。FastAPI `Security` / `Depends` で認証・認可強制。
4	MEDIUM	`common/security_vulnerability`	`Dockerfile:9`	ランタイムイメージ内の `pip@24.` に6件の CVE（MEDIUM 5 / LOW 1 `CVE-2026-13346` 等）。マルチステジビルドで本番から `pip` を除外。
5	MEDIUM	`common/security_vulnerability`	`api/routes.py:59`	**SEC-06 ライブDAST (`CWE-306`)**: 認証 `POST /v1/orders` が HTTP `Created`（`webhook_deferred`）で功。OAuth2/JWT 認証ガードを実装。
6	MEDIUM	`common/security_vulnerability`	`api/routes.py:60`	`POST /v1/orders` (L60) と `GET /v1/orders/{order_id}` (L83) が `has_auth_hint=false`。Bearer トクン検証と `customer_id` 所有権照を追加。
7	MEDIUM	`common/runtime_verification`	`api/routes.py:29`	実測カバレッジで `routes.py` 行 `56.0%`（分岐 `0.0%`）、`main.py` 行 `67.9%`。`with TestClient(create_app())` で Lifespan と全 HTTP 応答分岐を実行。
8	MEDIUM	`python/testing`	`tests/test_routes.py:23`	`router.routes` のパス文字列確認みで ASGI リクエスト未実行。`TestClient` による `200` / `201` / `404` / `422` 統合テスト追加。
9	LOW	`common/twelve_factor`	`main.py:33`	`InMemoryOrderRepository` がプロス内メモリ保持。マルチレプリカ

#	重大度	ルールID	箇所	検出された証跡と推奨修正内容
				用時は PostgreSQL/Redis 等の永続 `OrderRepository` 実装を選択可能する。
10	LOW	`python/frameworks/fastapi`	`infra/webhook_client.py:42`	通知毎に `httpx.AsyncClient` を生成・破棄。Lifespan 管理の共有 `AsyncClient` 注入で Keep-Alive 接プールを再利用。
11	LOW	`common/production_readiness`	`infra/webhook_client.py:43`	Webhook POST に `Idempotency-K` ヘッダや再送制御がない。`Idempotency-Key: record.order_` 付与と Outbox 再送を検討。
12	LOW	`common/supply_chain`	`workflows/ci.yml:18`	`ruff` と `pytest` のみ実行し、`pyproject.toml` 定義済の型チェッカ `ty` が未実行。`uv run ty chec` ステップを CI に追加。
13	LOW	`common/supply_chain`	`Dockerfile:14`	コンテナ `HEALTHCHECK` 未定義かつ `--host 127.0.0.1` 固定。オーケストレータ向けにホスト設定変数化 `/healthz` プローブを追加。
14	LOW	`common/documentation`	`README.md:14`	開発検証コマンド（`pytest` / `ruff` / `ty`）および `ORDER_DISPATCH_VIP_DISCOUNT_RA` が未記載。開発手順と環境変数表更新。
15	LOW	`common/security`	`settings.py:17`	`webhook_url: str` にスキーム検証がない。`HttpUrl`（本番 HTTPS 強制）および `http_timeout_sec` の限・下限 `Field` 制約を追加。
16	LOW	`common/security_vulnerability`	`Dockerfile:12`	Trivy IaC 指摘 `DS-0026`（`LOW`）: `Dockerfile` に `HEALTHCHECK` 命令未定義。Python 標準ライブラリによる `/healthz` 確認を追加。
17	LOW	`common/security_vulnerability`	`main.py:58`	**SEC-06 ライブDAST (`CWE-200`)**: `/openapi.json`（`4,753` bytes）が認証で公開。本番環境では `openapi_url=None` 等で非公開化。
18	LOW	`common/security_vulnerability`	`main.py:60`	**SEC-06 ライブDAST (`CWE-693`)**: `/healthz` 応答に `X-Content-Type-Options: nosniff` が未付与。ASGI セキュリティヘッダミドルウェアを追加。
19	LOW	`common/runtime_verification`	`infra/webhook_client.py:36`	実測カバレッジで `webhook_client.py` が `55.6%`（L L41-47 未通過）。

#	重大度	ルールID	箇所	検出された証跡と推奨修正内容
				`httpx.MockTransport` で 2xx・5xx・Timeout 経路を網羅。
20	**LOW**	`python/testing`	`tests/test_dispatcher.py:13`	`_StubWebhookClient` 差し替えに。り実クラス `AsyncWebhookClient` 単体テストが不在。`MockTranspor` テストを追加。

5. 詳細分析：コード品質・テスト・動的検証

静的 AST 解析（全14ファイル・435 LOC・23関数・14クラス）では、**型安全性** `100.0%`・**Docstring網羅率** `100.0%`・**Import衛生** `100.0%`、SIG 保守性リスク（複雑関数・過剰引数・循環依存・グローバル可変状態）すべて `0件` を確認しました。さらに Docker 隔離環境での動的検証（`run_check`）では、ビルド・オフライン単体テスト（`4/4` 成功）・ASGI `/healthz` スモーク（HTTP `200`）がすべて正常通過し、実測カバレッジは **行** `85.2%`（`178/209` **文）・分岐** `62.5%` を記録しました。

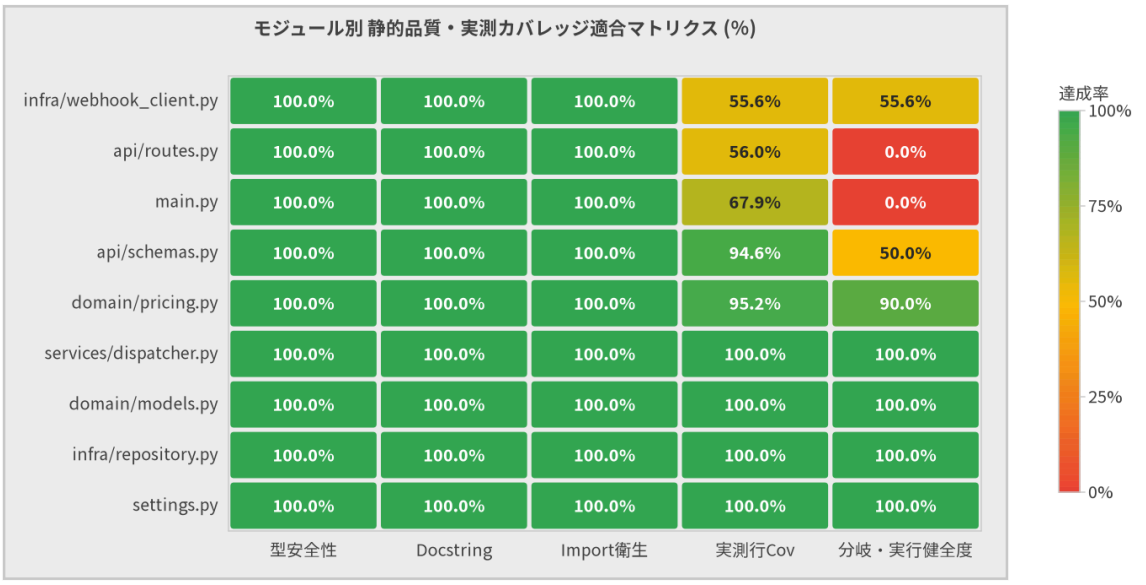


図5-1: 主要9モジュール × 5品質軸の健全性ヒートマップ（静的AST完全性 vs 実測行・分岐カバレッジ）

コンテナ動的検証ステージとモジュール別カバレッジ詳細

ドメイン層（`models.py` `100%`, `pricing.py` `95.2%`）およびサービス層（`dispatcher.py` `100%`）は単体テストで緻密に検証されている一方、リポジトリ内の `pytest` スイート単体では ASGI トランスポート層（`routes.py` `56.0%`, `main.py` `67.9%`）と外部 HTTP 通信アダプタ（`webhook_client.py` `55.6%`）が未実行行を残しています。なお、別工程の動的 Web ペネトレーションテスト

（`web_pentest`、`.pni/pentests/examples/python/api_fastapi_clean/`、`5.24s`）では稼働中の Uvicorn コンテナに対して計19件のライブ HTTP プローブを実行し、`/healthz`・`POST /v1/orders`（201 / 422）・`GET /v1/orders/1`（404）の実通信動作も補完確認しています。

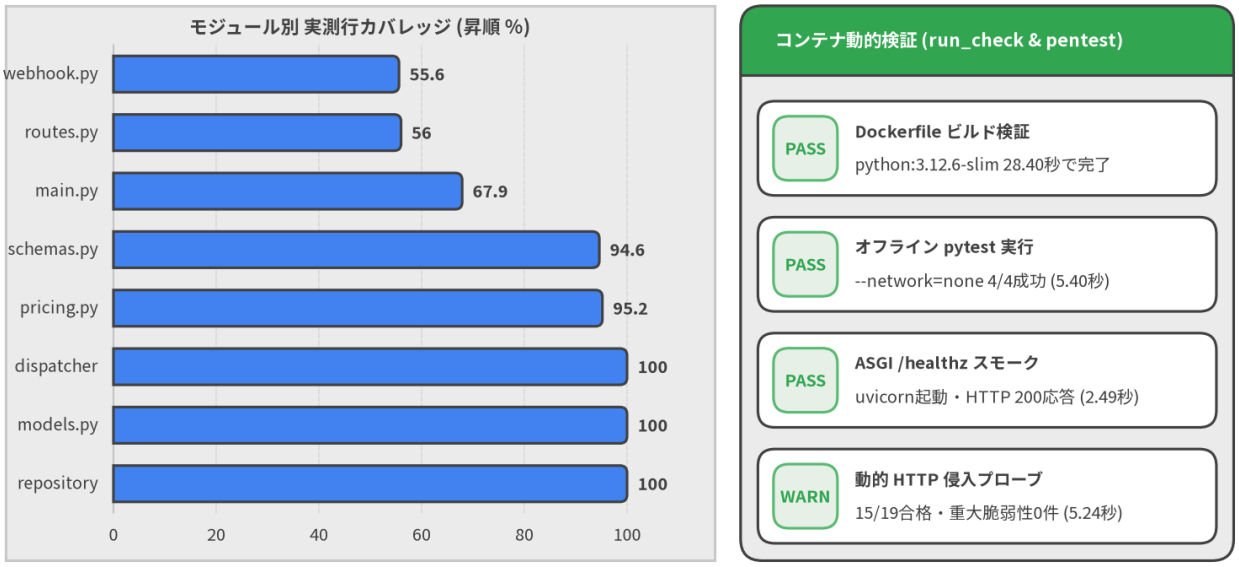


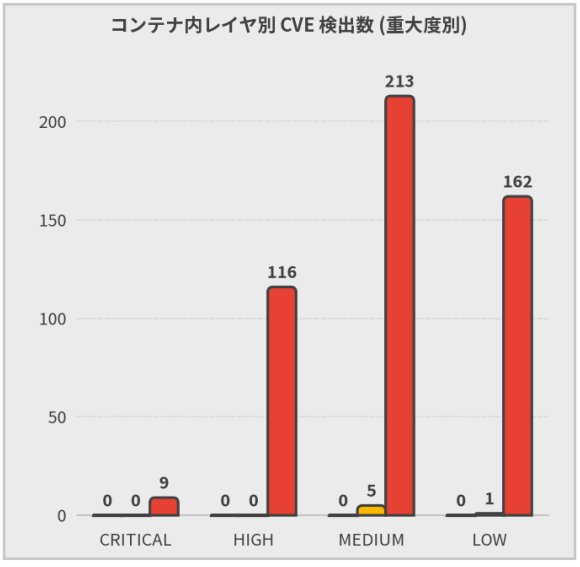
図5-2: モジュール別実測行カバレッジ比較（左）とコンテナ動的検証パイプライン結果（右）

静的品質指標・12-Factor 準備度と `pytest-cov` 実測内訳

モジュール相対パス	実行文 / 総文数	実測行カバレッジ	実測分岐カバレッジ	未通過行番号 (`missing_lines`) と補完テスト方針
`src/order_dispatch/infra/webhook_client.py`	`10 / 18`	**55.6%**	`100.0%` (0分岐)	`36, 41-47` ― `httpx.MockTransport` で 200 OK / 503 / `ConnectTimeout` を検証
`src/order_dispatch/api/routes.py`	`14 / 25`	**56.0%**	**0.0%**	`29-30, 42-43, 56, 74, 79, 96-98` ― `TestClient` で `/healthz`, `POST` / `GET /v1/orders` (201/200/404) を通電
`src/order_dispatch/main.py`	`19 / 28`	**67.9%**	**0.0%**	`30-34, 38, 43-45` ― `with TestClient(create_app()) as client:` で Lifespan 起動・終了を通電
`src/order_dispatch/api/schemas.py`	`35 / 37`	**94.6%**	`50.0%`	`32, 90` ― 空白 SKU バリデーション失敗（HTTP 422）と `to_order_response` 射影を検証
`src/order_dispatch/domain/pricing.py`	`20 / 21`	**95.2%**	`90.0%`	`49` ― 不正 `vip_discount_rate` 境界値に対する `PricingError` 例外テストを1件追加
`src/order_dispatch/services/dispatcher.py`	`26 / 26`	**100.0%**	`100.0%`	なし（正常配送 `DISPATCHED`・失敗縮退 `WEBHOOK_DEFERRED` とも検証済）
`src/order_dispatch/domain/models.py`	`24 / 24`	**100.0%**	`100.0%`	なし（全 `dataclass(frozen=True)`・`StrEnum` 網羅）
`src/order_dispatch/infra/repository.py`	`13 / 13`	**100.0%**	`100.0%`	なし（保存・ID検索・未存在 `None` 応答とも網羅）
`src/order_dispatch/settings.py` + `__init__.py` (5件)	`17 / 17`	**100.0%**	`100.0%`	なし（全設定フィールド・パッケージ初期化を通過）
合計 / 全体集計 (`pytest-cov`)	`178 / 209`	**85.2%**	**62.5%**	`TestClient` + `MockTransport` 約40行の追加で行・分岐とも `95%+` 到達可能

6. 詳細分析：セキュリティ・脆弱性・API境界

Trivy によるファイルシステムおよびビルド済コンテナイメージ検査（`security_scan`）、Python AST SAST 解析、および隔離ループバックコンテナに対する **動的 Web ペネトレーションテスト（`web_pentest` / `.pni/pentests/examples/python/api_fastapi_clean`）** の結果、**アプリケーションの直接・推移的 Python 実行時ライブラリ（`fastapi`, `httpx`, `pydantic`, `pydantic-settings`, `uvicorn`）の CVE は** `0件`、コード内の **CWE 危険シンクは** `0件`、秘密情報漏洩 `0件`、ライブ DAST における **CRITICAL / HIGH 脆弱性も** `0件`（`15/19` **プローブ合格）** でした。一方、コンテナ同梱 `pip@24.2` に6件（MEDIUM 5 / LOW 1）、古い `python:3.12.6-slim-bookworm`（Debian 12.7）OS 層に508件（CRITICAL 9 / HIGH 116）、およびライブ HTTP 検証で実証された3件の境界ハードニング課題（`CWE-306` 未認証 `POST /v1/orders` HTTP `201`、`CWE-200` `/openapi.json` 公開、`CWE-693` `nosniff` ヘッダ欠落）が確認されました。

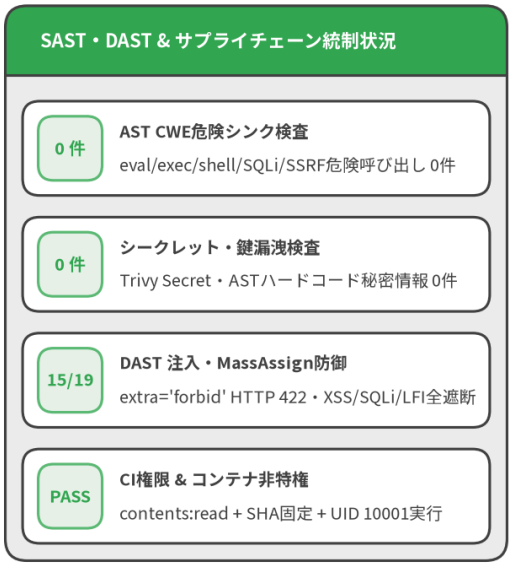


重大度	対象パッケージ	代表CVE / 修正済バージョン
CRIT	libgnutls30	CVE-2026-42010 -> 3.7.9-2+deb12u7
CRIT	libssl3 / openssl	CVE-2026-31789 -> 3.0.19-1~deb12u2
CRIT	perl-base	CVE-2026-8376 -> 5.36.0-7+deb12u4
MED	pip (24.2)	CVE-2026-13346他5件 -> 26.2.0 / 除外
LOW	Dockerfile	DS-0026 -> HEALTHCHECK 命令を追加

図6-1: レイヤ別CVE重大度内訳（左）と修正バージョンが判明している優先パッチ対象一覧（右）

AST 危険シンク検査（CWE）と HTTP API 攻撃対象領域マップ

AST セキュリティシンク走査ではコマンド実行・任意コード評価・SQLインジェクション・安全でないデシリアライズ等の CWE 危険パターンは一切検出されませんでした。外部公開前に強化すべき境界は、認証ヒントを持たない `/v1/orders` 系2エンドポイントの認証・テナント認可と、`main.py` における OpenAPI 公開制御・HTTP セキュリティヘッダ付与です。



HTTP エンドポイント	認証状態	ライブDAST挙動と推奨アクセス制御
GET /healthz (routes.py:L46)	不要 (公開可)	HTTP 200応答確認済。nosniff等のセキュリティヘッダ追加を推奨(CWE-693)
POST /v1/orders (routes.py:L59)	未実装 (CWE-306)	extra='forbid'で422拒否済だが未認証でHTTP 201成功。OAuth2/JWT認証を強制
GET /v1/orders/{id} (routes.py:L83)	未実装 (要対応)	未存在IDはHTTP 404返却(スタック漏洩無)。customer_id所有権の認可を追加

図6-2: サプライチェーン・SAST・DAST防御統制（左）とHTTP APIエンドポイント認可境界マップ（右）

動的Webペネトレーションテスト（`web_pentest` / `.pni/pentests/`）

隔離ループバック環境（`http://127.0.0.1:36095`、実行時間 `5.24s`）で稼働させた Uvicorn コンテナに対し、`.pni/pentests/examples/python/api_fastapi_clean/` の `pytest` + `httpx` プローブスイート（計4ファイル・全19プローブ）を実行しました。結果ステータスは `passed`（**合格** `15` / **失敗** `3` / **スキップ** `1` / **エラー** `0`、`CRITICAL` / `HIGH` **検出** `0件`）です。Pydantic v2 `ConfigDict(extra="forbid")` による未宣言フィールド（`total_cents`, `is_admin`）の `HTTP 422` 拒否や、反射型 XSS・SQL エラー／スタックトレース露出・パスト

ラバーサル・CORS Origin 反射・`.env` / `.git` 露出・HTTP `TRACE` への防御が実証された一方、以下の3件の改善項目がライブ HTTP 応答として裏付けられました。

結果 / 重大度	CWE / カテゴリ	プローブテスト (`test_file`)	対象エンドポ
FAIL (`MEDIUM`)	`CWE-306` (`missing_authentication`)	`test_create_order_enforces_authentication_guard` (`test_app_probes.py`)	`POST /v1/ord` (`src/order_d`
FAIL (`LOW`)	`CWE-200` (`api_schema_exposure`)	`test_openapi_schema_not_publicly_exposed_without_auth` (`test_app_probes.py`)	`GET /openapi` (`src/order_d`
FAIL (`LOW`)	`CWE-693` (`missing_security_headers`)	`test_x_content_type_options_nosniff` (`test_baseline_http.py`)	`GET /healthz` (`src/order_d`
PASS (15 件)	`CWE-915` , `79` , `89` , `22` , `209` , `538` , `942`	`test_app_probes.py` , `test_target_surface.py` , `test_baseline_http.py`	`/v1/orders` , `/.env` , `/.git`

主要 CVE・IaC 指摘一覧と修正バージョン対照表

区分	重大度	脆弱性 / ルールID	パッケージ（現在値）	修正バージョン（`fixed_version`）	概要と推奨緩和策
OS (Debian 12.7)	**CRITICAL**	`CVE-2026-42010`, `CVE-2026-33845`	`libgnutls30@3.7.9-2+deb12u3`	`3.7.9-2+deb12u7`	GnuTLS 認証バイパス / DTLS DoS。最新 `slim-bookworm` 取得・`apt-get upgrade` で解消。
OS (Debian 12.7)	**CRITICAL**	`CVE-2026-31789` (+ HIGH 2件)	`libssl3`, `openssl@3.0.14-1~deb12u2`	`3.0.19-1~deb12u2`（`3.0.20` 推奨）	X.509 証明書処理バッファオーバーフロー / PKCS7 UAF。OpenSSL セキュリティ更新で解消。
OS (Debian 12.7)	**CRITICAL**	`CVE-2026-8376`, `CVE-2026-42496`	`perl-base@5.36.0-7+deb12u1`	`5.36.0-7+deb12u4`	正規表現ヒープ破壊・`Archive::Tar` シンボリックリンク横断。Debian セキュリティ更新で解消。
OS (Debian 12.7)	**HIGH**	`CVE-2025-6965`	`libsqlite3-0@3.40.1-2`	`3.40.1-2+deb12u2`	SQLite 整数切り詰め脆弱性。最新ベースイメージ更新で解消（ランタイム未使用パッケージ）。
Python ツール	**MEDIUM**	`CVE-2026-13346`, `CVE-2026-8643` 他	`pip@24.2`（計6件: MED 5, LOW 1）	`26.2.0`（または本番除外）	マルチステージ `uv` ビルドにより本番イメージから `pip` 自体を除去することで根絶可能。
Python 実行時	**なし**	ー（`0 CVEs`）	`fastapi`, `httpx`, `pydantic`, `uvicorn`	対応不要（脆弱性ゼロ）	アプリケーションが直接利用する Python ランタイム依存パッケージに既知 CVE は存在しない。
IaC（`Dockerfile`）	**LOW**	`DS-0026`	`Dockerfile:L1-L14`	`HEALTHCHECK` 命令追加	`/healthz` を定期確認する `HEALTHCHECK` 命令を追加し、オーケストレータ自動復旧を有効化。